

BUỔI 8 - Routing, Đăng nhập, Axios Interceptor & Phân quyền Giao diện

Điều kiện tiên quyết: Đã hoàn thành Buổi 7 - CourseForm CRUD hoạt động qua Gateway, axiosClient đã có Request Interceptor bản tối giản đọc token từ localStorage.crs_token (dán tay qua Console).

Mục tiêu chính: Buổi này thay thế giải pháp tạm ở Buổi 7 bằng luồng đăng nhập thật, đồng thời tổ chức lại toàn bộ ứng dụng theo Route - từ nay App.tsx không còn là 1 trang duy nhất.

0. Mục tiêu buổi học

Sau buổi học này, sinh viên phải:

1. Cài và cấu hình react-router-dom, tách các màn hình đã làm (Buổi 6, 7) thành các trang riêng theo Route.
2. Viết trang /login gọi POST /api/auth/login, thay thế hoàn toàn thao tác "dán token bằng tay" của Buổi 7.
3. Viết AuthContext quản lý trạng thái đăng nhập toàn cục, ghi token vào đúng key localStorage.crs_token đã thống nhất từ Buổi 7 - không sửa axiosClient phần Request Interceptor.
4. Bổ sung Response Interceptor xử lý lỗi 401 (token hết hạn) → tự động đăng xuất.
5. Viết ProtectedRoute chặn /admin/courses nếu không phải ADMIN.
6. Ẩn nút quản trị (Sửa/Xoá/Thêm) trên giao diện CourseList/CourseForm khi người dùng là STUDENT - Role-Based UI.

A. Tổ chức lại ứng dụng theo Route

A.1. Vì sao phải tách trang bây giờ?

Từ Buổi 6-7, toàn bộ giao diện (danh sách + form CRUD) đều dồn chung vào App.tsx, không phân biệt "trang Admin quản lý môn học" và "trang Sinh viên xem môn học". Buổi sắp tới cần thêm trang "Đăng ký học phần" - nếu không tách route từ bây giờ, App.tsx sẽ phình to không kiểm soát được.

A.2. Cấu trúc route

Route	Trang	Ai được xem
/login	LoginPage	Công khai
/courses	CoursesPage	Công khai (ADMIN và STUDENT đều xem được danh sách)
/admin/courses	AdminCoursesPage	Chỉ ADMIN — chặn bằng ProtectedRoute
/register-course	RegisterCoursePage	Chỉ STUDENT đã đăng nhập (hoàn thiện đầy đủ ở Buổi sau, buổi này chỉ tạo khung trang)

A.3. Di chuyển code cũ vào đúng thư mục pages/

• Nội dung App.tsx của **Buổi 6** (chỉ có CourseList + SearchBox + Pagination, không có Form) → chuyển thành **src/pages/CoursesPage.tsx**.

- Nội dung App.tsx của **Buổi 7** (có thêm CourseForm CRUD) → chuyển thành **src/pages/AdminCoursesPage.tsx**.

```
// path: crs-frontend/src/pages/CoursesPage.tsx
// purpose: trang xem danh sach mon hoc cong khai, chuyen tu App.tsx
// của Buổi 6 sang,
// KHÔNG có Form Them/Sua/Xoa (danh cho cả ADMIN và STUDENT xem)

import { useState } from 'react';
import { useCourses } from '../api/useCourses';
import SearchBox from '../components/SearchBox';
import CourseList from '../components/CourseList';
import Pagination from '../components/Pagination';

export default function CoursesPage() {
  const [keyword, setKeyword] = useState('');
  const [page, setPage] = useState(0);
  const { courses, totalPages, state, errorMessage, refetch } =
    useCourses(keyword, page);

  const handleSearch = (newKeyword: string) => {
    setKeyword(newKeyword);
    setPage(0);
  };

  return (
    <div style={{ padding: 24, maxWidth: 800, margin: '0 auto' }}>
      <h1>Danh sach mon hoc</h1>
      <SearchBox onSearch={handleSearch} />
      <div style={{ marginTop: 16 }}>
        <CourseList
          courses={courses}
          state={state}
          errorMessage={errorMessage}
          onRetry={refetch}
        />
      </div>
      <Pagination currentPage={page} totalPages={totalPages}
onPageChange={setPage} />
    </div>
  );
}
```

```
// path: crs-frontend/src/pages/AdminCoursesPage.tsx
// purpose: rap CourseForm + CourseList + Pagination + SearchBox, xu
ly Them/Sua/Xoa
// va dong bo lai danh sach (refetch) sau moi thao tac thanh cong
// (chuyen tu App.tsx cua Buoi 7 sang, chi doi ten ham va export)
```

```
import { useState } from 'react';
import axios from 'axios';
import { useCourses } from '../api/useCourses';
import { createCourse, updateCourse, deleteCourse } from
'../api/courseApi';
import SearchBox from '../components/SearchBox';
import CourseList from '../components/CourseList';
import Pagination from '../components/Pagination';
import CourseForm from '../components/CourseForm';
import type { Course, CourseFormValues } from '../types/course';
import type { ApiErrorResponse } from '../types/apiError';

export default function AdminCoursesPage() {
  const [keyword, setKeyword] = useState('');
  const [page, setPage] = useState(0);
  const [editingCourse, setEditingCourse] = useState<Course |
null>(null);
  const [submitting, setSubmitting] = useState(false);
  const [formError, setFormError] = useState<string | null>(null);
  const { courses, totalPages, state, errorMessage, refetch } =
useCourses(keyword, page);

  const handleSearch = (newKeyword: string) => {
    setKeyword(newKeyword);
    setPage(0);
  };

  const extractErrorMessage = (err: unknown): string => {
    if (axios.isAxiosError<ApiErrorResponse>(err)) {
      const data = err.response?.data;
      if (data?.message) return data.message;
      if (data) {
        const firstFieldError = Object.values(data).find((v) =>
typeof v === 'string');
        if (firstFieldError) return firstFieldError;
      }
    }
  }
}
```

```

    return 'Da xay ra loi, vui long thu lai.';
  };

const handleFormSubmit = async (values: CourseFormValues) => {
  setSubmitting(true);
  setFormError(null);
  try {
    if (editingCourse) {
      await updateCourse(editingCourse.id, values);
    } else {
      await createCourse(values);
    }
    setEditingCourse(null);
    refetch();
  } catch (err) {
    setFormError(extractErrorMessage(err));
  } finally {
    setSubmitting(false);
  }
};

const handleDelete = async (course: Course) => {
  if (!window.confirm(`Xoa mon hoc "${course.tenMonHoc}"?`))
return;
  try {
    await deleteCourse(course.id);
    refetch();
  } catch (err) {
    alert(extractErrorMessage(err));
  }
};

return (
  <div style={{ padding: 24, fontFamily: 'sans-serif', maxWidth:
800, margin: '0 auto' }}>
    <h1>Quan ly mon hoc (Admin)</h1>
    <CourseForm
      editingCourse={editingCourse}
      onSubmit={handleFormSubmit}
      onCancel={() => setEditingCourse(null)}
      submitting={submitting}
      serverError={formError}
    />

```

```

    <SearchBox onSearch={handleSearch} />
    <div style={{ marginTop: 16 }}>
      <CourseList
        courses={courses}
        state={state}
        errorMessage={errorMessage}
        onRetry={refetch}
        onEdit={setEditingCourse}
        onDelete={handleDelete}
      />
    </div>
    <Pagination currentPage={page} totalPages={totalPages}
onPageChange={setPage} />
  </div>
);
}

```

Lưu ý quan trọng khi di chuyển code

CourseList dùng ở CoursesPage này không truyền onEdit/onDelete - sẽ cần cập nhật CourseList để 2 prop này trở thành tùy chọn (mục F bên dưới), vì trang công khai không có quyền sửa/xóa.

AdminCoursesPage.tsx giữ nguyên toàn bộ nội dung App.tsx của Buổi 7 (đổi tên file, đổi tên hàm từ App thành AdminCoursesPage, xóa export default App cũ) - không liệt lại code ở đây để tránh trùng lặp, sinh viên copy nguyên phần Buổi 7 sang.

B. AuthContext — quản lý trạng thái đăng nhập toàn cục

```

// path: crs-frontend/src/context/AuthContext.tsx
// purpose: quản lý trạng thái đăng nhập toàn cục, ghi/doc token
dùng key 'crs_token'
// đã thông nhất từ Buổi 7 - axiosClient KHÔNG cần sửa lại

import { createContext, useContext, useState, useEffect, type
ReactNode } from 'react';
import type { LoginResponse } from '../types/auth';

interface AuthUser {
  username: string;
  role: 'ADMIN' | 'STUDENT';
}

```

```

interface AuthContextValue {
  user: AuthUser | null;
  login: (data: LoginResponse) => void;
  logout: () => void;
  isAuthenticated: boolean;
}

const AuthContext = createContext<AuthContextValue |
undefined>(undefined);

const TOKEN_KEY = 'crs_token';
const USER_KEY = 'crs_user';

export function AuthProvider({ children }: { children: ReactNode })
{
  const [user, setUser] = useState<AuthUser | null>(null);

  // Khoi phuc phien dang nhap khi F5 trang (doc lai tu
localStorage)
  useEffect(() => {
    const savedUser = localStorage.getItem(USER_KEY);
    const savedToken = localStorage.getItem(TOKEN_KEY);
    if (savedUser && savedToken) {
      setUser(JSON.parse(savedUser));
    }
  }, []);

  const login = (data: LoginResponse) => {
    localStorage.setItem(TOKEN_KEY, data.token);
    const authUser: AuthUser = { username: data.username, role:
data.role };
    localStorage.setItem(USER_KEY, JSON.stringify(authUser));
    setUser(authUser);
  };

  const logout = () => {
    localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem(USER_KEY);
    setUser(null);
  };

  return (

```

```

    <AuthContext.Provider value={{ user, login, logout,
isAuthenticated: !!user }}>
      {children}
    </AuthContext.Provider>
  );
}

// eslint-disable-next-line react-refresh/only-export-components
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth phải được dùng bên trong
AuthProvider');
  return ctx;
}

```

Đối chiếu với Buổi 7

Hàm login() ghi vào localStorage.setItem('crs_token', ...) - đúng key mà axiosClient đã đọc từ Buổi 7 (mục B của Buổi 7). Đây chính là lý do vì sao axiosClient không cần sửa gì ở phần Request Interceptor tại buổi này.

C. Trang /login

```

// path: crs-frontend/src/api/authApi.ts
// purpose: hàm gọi POST /api/auth/login qua Gateway

import axiosClient from '../axiosClient';
import type { LoginRequest, LoginResponse } from '../types/auth';

export const login = (payload: LoginRequest) => {
  return axiosClient.post<LoginResponse>('/api/auth/login',
payload);
};

```

```

// path: crs-frontend/src/pages/LoginPage.tsx
// purpose: trang đăng nhập, gọi POST /api/auth/login, lưu vào
AuthContext rồi điều hướng

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';

```

```

import { login as loginApi } from '../api/authApi';
import { useAuth } from '../context/AuthContext';
import type { ApiErrorResponse } from '../types/apiError';

export default function LoginPage() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [submitting, setSubmitting] = useState(false);

  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    setSubmitting(true);
    try {
      const res = await loginApi({ username, password });
      login(res.data);
      navigate('/courses');
    } catch (err) {
      if (axios.isAxiosError<ApiErrorResponse>(err) &&
err.response?.data?.message) {
        setError(err.response.data.message);
      } else {
        setError('Dang nhap that bai, vui long thu lai.');

```



```

    <div style={{ marginBottom: 12 }}>
      <label>Mat khau</label><br />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        style={{ width: '100%' }}
      />
    </div>
    {error && <p style={{ color: '#b91c1c' }}>{error}</p>}
    <button type="submit" disabled={submitting} style={{ width:
'100%' }}>
      {submitting ? 'Dang xu ly...' : 'Dang nhap'}
    </button>
  </form>
</div>
);
}

```

D. Hoàn thiện Interceptor - Response Interceptor xử lý 401

axiosClient đã có Request Interceptor từ Buổi 7 (tự đính Authorization). Buổi này bổ sung Response Interceptor:

```

// path: crs-frontend/src/api/axiosClient.ts
// purpose: THEM Response Interceptor xu ly 401 (token het han/khong
hop le) -> tu dong dang xuat.
// Phan Request Interceptor GIU NGUYEN tu Buoì 7, khong sua.

import axios from 'axios';

const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request Interceptor - tu Buoì 7, giu nguyen
axiosClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('crs_token');
  if (token) {

```

```

    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response Interceptor - MÔI o Buổi 8
axiosClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (axios.isAxiosError(error) && error.response?.status === 401)
    {
      localStorage.removeItem('crs_token');
      localStorage.removeItem('crs_user');
      // Dùng window.location thay vì useNavigate() vì đây là file
      // thuần TypeScript,
      // không phải component - không thể dùng React Hook ở đây
      if (window.location.pathname !== '/login') {
        window.location.href = '/login';
      }
    }
    return Promise.reject(error);
  }
);

export default axiosClient;

```

Câu hỏi thảo luận

Tại sao interceptor bắt lỗi 401 lại chỉ tự đăng xuất, mà không xử lý luôn lỗi 403 (sai role)?

→ Vì 401 nghĩa là "chưa xác thực được / token hỏng", đăng xuất là hợp lý. Còn 403 nghĩa là "đã xác thực đúng nhưng không đủ quyền" (ví dụ STUDENT gọi API của ADMIN) - đây không phải lỗi phiên đăng nhập, không nên tự đăng xuất, chỉ cần báo lỗi cho người dùng (đã xử lý bằng extractErrorMessage ở Buổi 7).

E. ProtectedRoute và cấu hình Router

```

// path: crs-frontend/src/components/ProtectedRoute.tsx
// purpose: chặn truy cập route nếu chưa đăng nhập hoặc không đúng
// role yêu cầu

import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

```

```

interface ProtectedRouteProps {
  children: React.ReactNode;
  requiredRole?: 'ADMIN' | 'STUDENT';
}

export default function ProtectedRoute({ children, requiredRole }:
ProtectedRouteProps) {
  const { user, isAuthenticated } = useAuth();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (requiredRole && user?.role !== requiredRole) {
    return <Navigate to="/courses" replace />;
  }

  return <>{children}</>;
}

```

// path: crs-frontend/src/pages/RegisterCoursePage.tsx
 // purpose: khung trang tam thoi cho Buoi 9 - buoi nay chi tao route
 va bao ve bang ProtectedRoute

```

export default function RegisterCoursePage() {
  return (
    <div style={{ padding: 24 }}>
      <h1>Đăng ký học phần</h1>
      <p>(Sẽ hoàn thiện sau trong các buổi tiếp theo)</p>
    </div>
  );
}

```

// path: crs-frontend/src/App.tsx
 // purpose: khai bao toan bo Router cua ung dung, thay the noi dung
 App.tsx cu cua Buoi 6-7

```

import { BrowserRouter, Routes, Route, Navigate } from 'react-

```

```

router-dom';
import { AuthProvider } from '../context/AuthContext';
import ProtectedRoute from '../components/ProtectedRoute';
import LoginPage from '../pages/LoginPage';
import CoursesPage from '../pages/CoursesPage';
import AdminCoursesPage from '../pages/AdminCoursesPage';
import RegisterCoursePage from '../pages/RegisterCoursePage';
import Navbar from '../components/Navbar';

function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Navbar />
        <Routes>
          <Route path="/" element={<Navigate to="/courses" replace
/>} />
          <Route path="/login" element={<LoginPage />} />
          <Route path="/courses" element={<CoursesPage />} />
          <Route
            path="/admin/courses"
            element={
              <ProtectedRoute requiredRole="ADMIN">
                <AdminCoursesPage />
              </ProtectedRoute>
            }
          />
          <Route
            path="/register-course"
            element={
              <ProtectedRoute requiredRole="STUDENT">
                <RegisterCoursePage />
              </ProtectedRoute>
            }
          />
        </Routes>
      </AuthProvider>
    </BrowserRouter>
  );
}

export default App;

```

E.1. Navbar - điều hướng + nút Đăng xuất + ẩn/hiện menu theo Role

```
// path: crs-frontend/src/components/Navbar.tsx
// purpose: thanh dieu huong, hien thi menu khac nhau tuy theo trang
thai dang nhap va role

import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function Navbar() {
  const { user, isAuthenticated, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  return (
    <nav style={{ display: 'flex', gap: 16, padding: 12,
borderBottom: '1px solid #ddd', alignItems: 'center' }}>
      <Link to="/courses">Danh sach mon hoc</Link>

      {isAuthenticated && user?.role === 'ADMIN' && (
        <Link to="/admin/courses">Quan tri mon hoc</Link>
      )}

      {isAuthenticated && user?.role === 'STUDENT' && (
        <Link to="/register-course">Dang ky hoc phan</Link>
      )}

      <div style={{ marginLeft: 'auto' }}>
        {isAuthenticated ? (
          <>
            <span style={{ marginRight: 12 }}>Xin chao,
{user?.username} ({user?.role})</span>
            <button onClick={handleLogout}>Dang xuat</button>
          </>
        ) : (
          <Link to="/login">Dang nhap</Link>
        )}
      </div>
    </nav>
  );
}
```

```
    </nav>
  );
}
```

F. Role-Based UI trong CourseList

Cập nhật CourseList (đã viết ở Buổi 6-7) để onEdit/onDelete trở thành tùy chọn - CoursesPage (trang công khai) không truyền vào, AdminCoursesPage vẫn truyền như cũ:

```
// path: crs-frontend/src/components/CourseList.tsx
// purpose: cap nhat onEdit/onDelete thanh optional, chi hien cot
// "Thao tac" khi duoc truyen vao

import type { Course } from '../types/course';
import type { LoadState } from '../api/useCourses';

interface CourseListProps {
  courses: Course[];
  state: LoadState;
  errorMessage: string;
  onRetry: () => void;
  onEdit?: (course: Course) => void;
  onDelete?: (course: Course) => void;
}

export default function CourseList({
  courses, state, errorMessage, onRetry, onEdit, onDelete,
}: CourseListProps) {
  if (state === 'loading') return <p>Dang tai danh sach mon
hoc...</p>;

  if (state === 'error') {
    return (
      <div style={{ color: '#b91c1c' }}>
        <p>{errorMessage}</p>
        <button onClick={onRetry}>Thu lai</button>
      </div>
    );
  }

  if (state === 'empty') return <p>Khong tim thay mon hoc nao phu
hop.</p>;
```

```

const showActions = !!onEdit || !!onDelete;

return (
  <table style={{ width: '100%', borderCollapse: 'collapse' }}>
    <thead>
      <tr style={{ textAlign: 'left', borderBottom: '2px solid #333' }}>
        <th>Ten mon hoc</th>
        <th>So tin chi</th>
        <th>So cho con lai</th>
        {showActions && <th>Thao tac</th>}
      </tr>
    </thead>
    <tbody>
      {courses.map((course) => (
        <tr key={course.id} style={{ borderBottom: '1px solid #eee' }}>
          <td>{course.tenMonHoc}</td>
          <td>{course.soTinChi}</td>
          <td style={{ color: course.soChoConLai === 0 ? '#b91c1c' : 'inherit' }}>
            {course.soChoConLai} / {course.soChoToiDa}
          </td>
          {showActions && (
            <td>
              {onEdit && <button onClick={() => onEdit(course)}>Sua</button>}
              {onDelete && (
                <button onClick={() => onDelete(course)} style={{
marginLeft: 8, color: '#b91c1c' }}>
                  Xoa
                </button>
              )}
            </td>
          )}
        </tr>
      )})}
    </tbody>
  </table>
);
}

```

Nhắc nhở quan trọng về Bảo mật

Ấn nút trên giao diện (Role-Based UI) chỉ giúp trải nghiệm người dùng gọn gàng, KHÔNG PHẢI lớp bảo mật thật. Lớp bảo mật thật là SecurityConfig với hasRole("ADMIN") đã viết ở course-service (Buổi 4). Dù STUDENT có tự sửa code JS bằng DevTools để hiện lại nút Sửa/Xoá, request gửi lên vẫn sẽ bị course-service từ chối 403.

G. Test theo kịch bản đầy đủ

Xoá thủ công localStorage.crs_token (nếu còn từ Buổi 7) trước khi bắt đầu để test đúng luồng thật:

#	Thao tác	Kỳ vọng
1	Mở /admin/courses khi chưa đăng nhập	Tự chuyển hướng về /login
2	Đăng nhập sai mật khẩu	Hiện lỗi "Sai username hoặc password"
3	Đăng nhập đúng student1/student123	Chuyển tới /courses, Navbar hiện "Xin chào, student1 (STUDENT)" và menu "Đăng ký học phan"
4	Với tài khoản student1, cố tình gõ tay URL /admin/courses	Tự chuyển hướng về /courses (không phải /login, vì đã đăng nhập nhưng sai role)
5	Đăng xuất, đăng nhập lại bằng admin/admin123	Navbar hiện menu "Quan tri mon hoc", vào /admin/courses xem/sửa/xoá bình thường
6	F5 lại trang khi đang đăng nhập	Vẫn giữ nguyên trạng thái đăng nhập (không bị đá về /login) nhờ useEffect khôi phục phiên trong AuthContext
7	Tại /admin/courses, tự tay sửa localStorage.crs_token thành 1 chuỗi rác bất kỳ, sau đó thử bấm "Thêm mới"	course-service trả 401 → Response Interceptor tự động đăng xuất, chuyển về /login
8	Ở trang /courses (không đăng nhập), kiểm tra CourseList	Không hiện cột "Thao tác"

H. Git commit

```
cd crs-microservices
git add crs-frontend/
git commit -m "feat: routing + login via auth-service + interceptor + protected route"
git push
```

Sản phẩm đầu ra cuối buổi

- ☒ react-router-dom hoạt động đúng với 4 route: /login, /courses, /admin/courses, /register-course.
- ☒ Đăng nhập thật qua POST /api/auth/login, không còn thao tác dán token bằng tay của Buổi 7.

- ☑ AuthContext giữ đúng trạng thái đăng nhập, khôi phục lại sau khi F5 trang.
- ☑ Request Interceptor (từ Buổi 7) và Response Interceptor (mới, xử lý 401) hoạt động đúng, không cần sửa lại phần code cũ.
- ☑ ProtectedRoute chặn đúng /admin/courses với người chưa đăng nhập lẫn người sai role.
- ☑ CourseList ẩn đúng cột "Thao tác" tại trang công khai /courses.
- ☑ Toàn bộ luồng vẫn chỉ giao tiếp qua Gateway - sinh viên/Frontend không cần biết auth-service chạy ở đâu.
- ☑ Git có commit feat: routing + login via auth-service + interceptor + protected route.

Lỗi thường gặp & cách xử lý nhanh

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Đăng nhập thành công nhưng F5 lại bị đá về /login	Thiếu useEffect khôi phục phiên trong AuthContext, hoặc đọc sai key localStorage	Xem lại mục B
useAuth() báo lỗi "phai duoc dung ben trong AuthProvider"	Component gọi useAuth() nằm ngoài <AuthProvider> trong App.tsx	Kiểm tra lại thứ tự bọc component ở mục E
Đăng nhập xong, axiosClient vẫn không gửi kèm token	AuthContext.login() ghi sai tên key so với key mà Request Interceptor của Buổi 7 đang đọc	Đối chiếu lại đúng chuỗi 'crs_token' ở cả 2 nơi
ProtectedRoute luôn chuyển hướng dù đã đăng nhập đúng role	So sánh user?.role !== requiredRole bị lệch chữ hoa/thường ('admin' khác 'ADMIN')	Kiểm tra lại giá trị role trả về từ auth-service (Buổi 4) luôn là chữ HOA
Bấm "Dang xuat" nhưng vẫn còn gọi được API có quyền ADMIN	Quên logout() cũng phải xóa cả crs_user, không chỉ crs_token	Xem lại mục B, hàm logout
Test case 7 (token rác) không tự đăng xuất	Chưa thêm đúng Response Interceptor, hoặc thêm nhầm vào courseApi.ts thay vì axiosClient.ts	Xem lại mục D